

# News-Hash: Konstruktion eines nachvollziehbaren Nachrichtenarchivs

Ein wissenschaftlicher Projektbericht zu Datenmodell, Integritätsprüfung, Quellenverarbeitung und öffentlicher Zeitverankerung

News-Hash-Projekt · Technischer Bericht · 17. August 2026

**Transparenzhinweis zur Entstehung:** Sowohl die News-Hash-App als auch dieser wissenschaftliche Artikel wurden fast vollständig KI-generiert. Die menschliche Mitwirkung bestand insbesondere in Zielsetzung, Anweisungen, Prüfung, Auswahl und Freigabe der Ergebnisse.

*„Mit KI gebaut, mit Liebe verfeinert, für neugierige Menschen.“*

**Zusammenfassung.** Die digitale Nachrichtenüberlieferung ist durch nachträgliche Redaktion, wechselnde URLs und das Verschwinden einzelner Veröffentlichungen geprägt. Der vorliegende Beitrag beschreibt News-Hash als ein lokal betriebenes Nachrichtenarchiv, das konfigurierte JSON- und XML-Quellen regelmäßig abrufen, normalisiert und in zwei append-only Speicherformaten ablegt. JSONL und SQLite führen unabhängige Hash-Ketten, sodass Änderungen am Inhalt oder an der Reihenfolge nachträglich erkennbar werden. Ein tägliches Manifest verdichtet die jeweiligen Kettenenden und kann mit OpenTimestamps öffentlich zeitverankert werden, ohne Nachrichtentexte an einen externen Dienst zu übertragen. Untersucht werden die Architektur, das Datenmodell, die Codec-Schnittstelle, die Fehlersemantik, die WebUI sowie das Vertrauens- und Bedrohungsmodell. Der Beitrag argumentiert, dass die Kombination etablierter Verfahren eine belastbare technische Herkunftsspur erzeugt, ohne den Anspruch zu erheben, journalistische Wahrheit, Quellenvollständigkeit oder politische Neutralität zu beweisen.

**Schlüsselwörter:** Nachrichtenarchivierung, digitale Provenienz, Hash-Kette, SHA-256, OpenTimestamps, Datenintegrität

## 1. Einleitung

Nachrichten sind ein zentraler Bestandteil öffentlicher Wissensbildung und zugleich ein ungewöhnlich flüchtiger Informationstyp. Redaktionen aktualisieren Texte, korrigieren Überschriften, tauschen Bilder aus oder entfernen Beiträge. Suchmaschinen und soziale Plattformen bewahren häufig nur Verweise auf eine jeweils aktuelle Fassung. Für eine spätere Analyse entsteht dadurch ein Quellenproblem: Die heute erreichbare Seite muss nicht der Version entsprechen, die eine Person gestern gelesen und auf deren Grundlage sie gehandelt hat. Das Problem entspricht der allgemeinen Herausforderung digitaler Langzeitarchivierung, die das

OAIS-Referenzmodell als Zusammenspiel von Übernahme, Erhaltung, Verwaltung und Zugriff beschreibt [6].

Ein Archiv kann auf diese Flüchtigkeit unterschiedlich reagieren. Es kann versuchen, möglichst viele Ressourcen zu kopieren, oder es kann die konkrete lokale Repräsentation so dokumentieren, dass ihre spätere Unverändertheit geprüft werden kann. News-Hash verfolgt den zweiten Schwerpunkt. Die Anwendung liest konfigurierte Quellen regelmäßig ein, erkennt bereits bekannte Meldungen, speichert neue Datensätze dauerhaft und verknüpft jeden Datensatz mit dem Hash seines Vorgängers. Dadurch wird aus einer Folge von Abrufen eine nachvollziehbare Archivspur.

Die technische Konstruktion ist bewusst pragmatisch. Sie benötigt keine eigene Blockchain und keinen permanenten externen Konsens. Die Inhalte bleiben lokal in JSONL- und SQLite-Dateien. Erst die Hashwerte der lokalen Ketten werden in einem kleinen Manifest zusammengefasst und optional öffentlich mit OpenTimestamps verankert. Die externe Infrastruktur erhält somit keinen Nachrichtenbestand, sondern nur einen kryptografischen Fingerabdruck eines lokalen Zustands.

Der Artikel behandelt News-Hash als Systementwurf und nicht als empirische Studie über Medienqualität. Im Mittelpunkt stehen die Fragen, welche Integritätsaussagen die Architektur tatsächlich ermöglicht, welche Betriebsannahmen dafür erforderlich sind und an welchen Stellen eine Interpretation über die technische Evidenz hinausgehen würde.

## **2. Forschungsziel und Leitfragen**

Das Forschungsziel ist die Konstruktion eines kleinen, reproduzierbar betreibbaren Archivs für laufend aktualisierte Nachrichtenquellen. „Reproduzierbar“ bedeutet dabei nicht, dass jeder externe Webinhalt unter allen Umständen bitgleich erneut erzeugt werden kann. Gemeint ist vielmehr, dass Datenmodell, Hashmaterial, Speicherschritte, Konfiguration und Prüfregele so beschrieben sind, dass ein Dritter die zentrale Integritätsaussage nachvollziehen kann.

Aus diesem Ziel ergeben sich fünf Leitfragen. Erstens: Wie muss ein normalisierter Datensatz aufgebaut sein, damit seine Hashberechnung unabhängig von der Eingabereihenfolge stabil bleibt? Zweitens: Wie können ein zeilenorientiertes Dateiformat und eine relationale Datenbank parallel betrieben werden, ohne eine unklare gemeinsame Transaktionsgrenze vorzutäuschen? Drittens: Wie lassen sich quellspezifische Anforderungen, etwa vollständige Artikeltexte oder Screenshots, in einen allgemeinen Importprozess integrieren? Viertens: Wie können Fehler einzelner Quellen isoliert werden, ohne den gesamten Daemon zu stoppen? Fünftens: Wie lässt sich die Existenz eines lokalen Kettenzustands unabhängig vom Betreiber zeitlich einordnen?

Die Arbeit nutzt einen konstruktionsorientierten Ansatz. Anforderungen werden in Invarianten übersetzt und anschließend in Datenmodell, Codecs, Speicherlogik, WebUI und Tests umgesetzt. Die Untersuchung betrachtet Normalbetrieb, Wiederholungsabruf, beschädigte Eingaben, Prozessabbruch und nachträgliche Änderung eines Datensatzes. Entscheidend ist dabei nicht nur, was das System erkennt, sondern auch, welche Aussagen es nicht treffen kann.

### **3. Begriffliche Einordnung**

News-Hash verbindet drei Ebenen, die in der Diskussion über digitale Archive häufig getrennt betrachtet werden. Die erste Ebene ist die lokale Inhaltsbewahrung. Sie beantwortet die Frage, welche konkreten Felder, Bilder und Zusatzinformationen gespeichert wurden. Die zweite Ebene ist die interne Nachvollziehbarkeit. Hash-Ketten zeigen, ob eine gespeicherte Folge seit ihrer Erzeugung verändert wurde. Die dritte Ebene ist der externe Existenznachweis. Ein OpenTimestamps-Proof kann bestätigen, dass ein bestimmter Hash spätestens zu einem Zeitpunkt existierte. In der Provenienzforschung wird eine solche Verbindung von Entitäten, Aktivitäten und verantwortlichen Akteuren als Grundlage für Qualitäts- und Vertrauensbewertungen verstanden [7].

Diese Ebenen dürfen nicht miteinander verwechselt werden. Ein Webarchiv kann umfangreiche Ressourcen speichern und dennoch keine nachträgliche Änderung sichtbar machen. Eine Hash-Kette kann Integrität prüfen, ohne die Quelle vollständig zu kopieren. Ein Zeitstempel kann die Existenz eines Hashes belegen, ohne den zugehörigen Nachrichtentext zu veröffentlichen oder dessen Richtigkeit zu bewerten. Die Stärke des Projekts liegt in der bewussten Kombination, nicht in einer einzelnen Komponente. Für den Zugriff auf frühere Zustände von Webressourcen beschreibt das Memento-Framework mit Datetime Negotiation und TimeMaps ein komplementäres Modell [8]; News-Hash speichert dagegen seine beobachteten Zustände lokal und verknüpft sie kryptografisch.

Ebenso ist die interne Kette keine Blockchain im engeren Sinn. Eine Blockchain setzt typischerweise verteilte Replikation, Konsens und ein Modell gegen konkurrierende Zustände voraus. News-Hash arbeitet zunächst mit einem lokalen Betreiber und einer lokalen Datenhaltung. Die öffentliche Zeitverankerung wird nachgelagert eingesetzt. Diese Entscheidung reduziert Komplexität, macht aber die Vertrauensannahme sichtbar: Der Betreiber ist für Abruf, Konfiguration und Veröffentlichung der Daten verantwortlich.

### **4. Gesamtarchitektur**

Die Anwendung besteht aus einem CLI-Programm, einer Import- und Normalisierungsschicht, einer Codec-Schnittstelle, zwei Speicherschichten, einem optionalen Anchoring-Prozess und einer eingebetteten HTTP-WebUI. Die Quellen werden in `data/settings.toml` beschrieben. Jeder

Quellenblock enthält einen Anzeigenamen, eine Feed-URL, einen Speichernamen, einen Codecnamen und ein individuelles Polling-Intervall. Die Verarbeitung läuft quellenbezogen, damit ein Fehler bei einem Anbieter andere Anbieter nicht blockiert.

Der Daemon übernimmt die zeitgesteuerte Ausführung. Ein einmaliger Lauf ruft jede konfigurierte Quelle einmal ab; im Daemon-Modus wird jede Quelle nach ihrem eigenen Intervall erneut verarbeitet. Die Importlogik führt Abruf, Parsing, Deduplizierung, Normalisierung und Speicherung aus. Laufzeitfehler werden pro Quelle gezählt, nach `stderr` geschrieben und im Prozessspeicher für Dashboard und Prometheus-Metriken vorgehalten.

Die WebUI ist kein separates Frontend-Deployment, sondern Teil des Prozesses. Sie greift auf SQLite zu, zeigt Quellenkarten, Kennzahlen, Meldungslisten und Detailansichten und stellt Metriken unter `/metrics` bereit. Diese enge Kopplung vereinfacht die Installation eines lokalen Archivs. Zugleich ist sie eine Sicherheitsgrenze: Die WebUI hat keine Benutzerverwaltung und keine Authentifizierung und darf daher nicht ungeschützt im öffentlichen Netz betrieben werden.

Die Architektur trennt bewusst fachliche und technische Verantwortlichkeiten. Die Normalisierung entscheidet, welche Werte ein Datensatz besitzt. Der Hash-Codec entscheidet, wie diese Werte kanonisch verarbeitet werden. JSONL und SQLite entscheiden, wie sie dauerhaft abgelegt werden. Anchoring und GitHub-Synchronisierung arbeiten erst auf den Kettenendpunkten. Diese Schichtung erlaubt eine unabhängige Prüfung und begrenzt Seiteneffekte bei Erweiterungen.

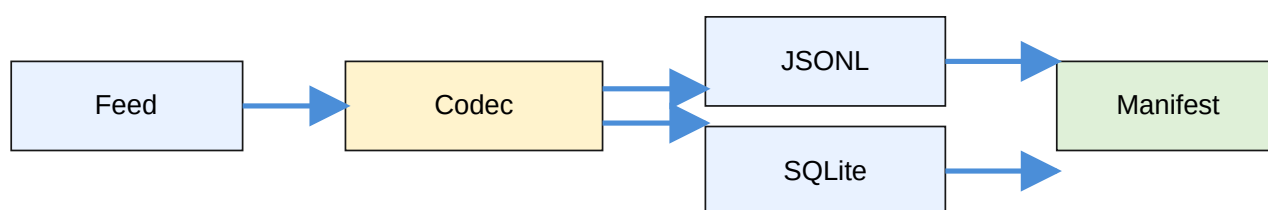


Abbildung 1: Vereinfachter Datenfluss von der Quelle bis zum täglichen Manifest.

## 5. Quellenaufnahme und Normalisierung

Ein Quellenlauf beginnt mit dem Abruf eines JSON- oder XML-Feeds über HTTP. Das externe Format wird in eine interne Feedrepräsentation überführt. Dabei werden Titel, URL, Inhalt, Autor, Veröffentlichungszeit und Bildinformationen aus unterschiedlichen Eingabefeldern zusammengeführt. Zeitangaben werden intern als UTC-Werte behandelt und erst in der Browserdarstellung lokal interpretiert. Die Normalisierung ist nicht nur eine Komfortfunktion; sie definiert das fachliche Material, das später gehasht wird.

Vor teuren Verarbeitungsschritten prüft die Anwendung bereits bekannte `source_id`-Werte. Diese Deduplizierung reduziert Netzwerkzugriffe, Bilddownloads und unnötige Kettenelemente. Die Entscheidung setzt allerdings voraus, dass die Kennung einer Quelle stabil und semantisch sinnvoll ist. Eine Quelle, die dieselbe Kennung für mehrere Fassungen verwendet, kann durch die Deduplizierungsregel relevante Änderungen verbergen; eine Quelle mit instabilen Kennungen kann dagegen unnötige Duplikate erzeugen.

Die Codec-Schnittstelle kapselt diese Besonderheiten. `rssv0` verarbeitet allgemeine JSON-Feeds und klassischen XML-RSS. `tazv0` lädt zusätzlich den vollständigen Artikel über den Feed-Link, wenn der Feed nur eine verkürzte Darstellung enthält. `screenv0` öffnet den ersten Link mit Chromium, wartet auf die Darstellung, entfernt bekannte Consent- und Cookie-Banner und speichert einen vollständigen PNG-Screenshot. Der Screenshot konserviert eine visuelle Momentaufnahme, nicht nur die extrahierten Textfelder.

Mit größerer Quellenflexibilität wachsen die dokumentarischen Unsicherheiten. Externe Seiten können dynamische Inhalte, personalisierte Elemente, nachgeladene Bilder oder wechselnde Werbeflächen enthalten. Ein Screenshot hängt von Browser, Viewport und Zeitpunkt ab. News-Hash behauptet daher nicht, eine Quelle vollständig oder dauerhaft abzubilden. Es archiviert die Fassung, die unter den gegebenen Abrufbedingungen beobachtet wurde.

## **6. Persistenz in JSONL und SQLite**

JSONL und SQLite erfüllen komplementäre Funktionen. JSONL ist ein einfaches Archivformat: Jede Zeile enthält einen JSON-Datensatz, der mit Standardwerkzeugen gelesen, kopiert und versioniert werden kann. SQLite stellt Indizes, Abfragen und relationale Verknüpfungen für die WebUI bereit. Bilder werden für JSONL als Dateien unter `data/images/` referenziert und in SQLite zusätzlich als deduplizierte BLOB-Daten abgelegt.

Beide Speicher führen eine eigene Hash-Kette. Der letzte bekannte Hash, die bekannten Quellenkennungen und der Vorgängerwert werden pro Speicherformat ermittelt. Diese Entscheidung ist eine direkte Reaktion auf die unterschiedliche Dauerhaftigkeit der beiden Schreibwege. Ein Prozess kann zwischen einem Dateischreibvorgang und einer SQLite-Transaktion beendet werden. Anstatt eine nicht vorhandene globale Transaktion zu simulieren, macht die Anwendung die beiden Zustände unabhängig prüfbar.

Erreicht ein Shard eine Größe von 1 GB, wird eine neue nummerierte Datei begonnen. Die Dashboard-Vorschau liest aus Performancegründen nur den neuesten SQLite-Shard. Aggregierte Kennzahlen und Detailzugriffe können dagegen über alle Shards arbeiten. Für die Deduplizierung wird ebenfalls nur der neueste Shard je Speicherformat betrachtet. Diese Regeln

sind ein Kompromiss zwischen Importgeschwindigkeit, Speicherwachstum und historischer Abfragefähigkeit.

Bei einer Schemaabweichung wird eine vorhandene SQLite-Tabelle nicht automatisch gelöscht. Sie wird nach einem Namen mit Zeitstempel umbenannt und durch eine Tabelle mit aktuellem Schema ersetzt. Dadurch bleibt der alte Zustand forensisch verfügbar. Eine vollständige Migration oder semantische Zusammenführung ist bewusst nicht Teil dieser Operation; sie würde die Gefahr erhöhen, historische Daten während des Programmstarts irreversibel zu verändern.

## 7. Formale Definition der Hash-Kette

Sei  $R_i$  ein normalisierter Datensatz und  $H_i$  sein gespeicherter Hash. Der erste Datensatz erhält als Vorgängerwert einen String aus 64 Nullen. Für jeden weiteren Datensatz wird der Hash des unmittelbar vorherigen Datensatzes als  $P_i$  übernommen. Das Hashmaterial besteht aus den fachlichen Feldern und diesem Vorgängerwert. Es wird als JSON-Objekt mit sortierten Schlüsseln, ohne zusätzliche Leerzeichen und in UTF-8 serialisiert. Die Wahl von SHA-256 folgt einem etablierten Standard für Message-Digest-Verfahren, deren Zweck unter anderem die Erkennung von Änderungen an Nachrichten seit der Digest-Erzeugung ist [9].

```

$$P_0 = 0^{64}$$

$$H_i = \text{SHA-256}(\text{canonical\_json}(R_i, P_i))$$

$$P_{i+1} = H_i$$

```



Abbildung 2: Jeder Datensatz übernimmt den Hash des Vorgängers als Teil seines Hashmaterials.

Für den Standardcodec umfasst das Material unter anderem Autor, Inhalt, Codecname, Bilder, Vorgängerhash, Veröffentlichungszeit, Quellenkennung, Quell-URL und Titel. Der Abrufzeitpunkt wird gespeichert, fließt aber nicht in den Inhalts-Hash ein. Damit erzeugt ein späterer Abruf eines unveränderten veröffentlichten Inhalts nicht allein wegen des neuen Abrufzeitpunkts einen anderen Hash.

Die Validierung prüft zwei Beziehungen. Erstens muss der gespeicherte Vorgängerhash zum erwarteten Datensatz passen. Zweitens muss der gespeicherte Hash dem Ergebnis der erneuten Berechnung entsprechen. Eine Inhaltsänderung verletzt die zweite Beziehung; eine Umordnung oder das Einfügen an einer unerwarteten Stelle verletzt die Kettenbeziehung. Änderungen an

einem frühen Datensatz wirken sich durch die Folge der Vorgängerwerte auf spätere Datensätze aus.

Das Modell hängt von einer stabilen kanonischen Repräsentation ab. Eine Änderung am Feldmapping, an der Serialisierung oder am Algorithmus kann daher nicht ohne Weiteres als normale Datenänderung behandelt werden. Für eine langfristige Weiterentwicklung wären explizite Schema- und Algorithmusversionen im Datensatz oder Manifest sinnvoll. Der gegenwärtige Entwurf dokumentiert die Feldzuordnung im Codec und deckt die Berechnung durch Tests ab.

## **8. Fehler, Abstürze und Wiederanlauf**

Ein Archivdienst muss nicht nur den Erfolgsfall, sondern auch unvollständige und widersprüchliche Zustände beschreiben. Netzwerkfehler, ungültiges JSON, nicht erreichbare Artikel, fehlende Bilder und Browserprobleme werden pro Quelle behandelt. Die betroffene Quelle erhält einen Laufzeitfehler; andere Quellen laufen weiter. Die Fehlermeldung wird nicht Teil der Hash-Kette, weil sie ein flüchtiger Betriebszustand und kein archivierter Nachrichteninhalt ist.

Beim Prozessabbruch zwischen zwei Speicheroperationen kann JSONL bereits den neuen Datensatz enthalten, während SQLite noch auf dem vorherigen Kettenende steht. Die beiden Ketten sind dann nicht gleich lang, aber nicht deshalb automatisch ungültig. Ein Wiederanlauf muss die jeweils letzte gültige Kette fortsetzen. Die getrennte Verwaltung macht sichtbar, welcher Speicher voraus ist, und verhindert, dass eine beschädigte Seite die andere stillschweigend überschreibt.

Ein Neustart setzt die flüchtigen Fehlerzähler und das Laufzeitprotokoll zurück. Persistierte Datensätze, Bilder, Shards und Anchor-Dateien bleiben dagegen erhalten. Diese Unterscheidung ist für die Interpretation des Dashboards wichtig. Ein leerer Fehlerzähler bedeutet nicht, dass die Quelle historisch immer erfolgreich war; er bedeutet nur, dass der aktuelle Prozess seit seinem Start keine entsprechende Beobachtung gespeichert hat.

Die Fehlersemantik ist damit absichtlich konservativ. News-Hash versucht nicht, eine beschädigte Quelle automatisch zu reparieren oder historische Ketten nachträglich zu glätten. Eine Reparatur kann als neue technische Operation dokumentiert werden, aber sie darf nicht mit einer unveränderten ursprünglichen Archivspur verwechselt werden. Diese Haltung erhöht die Nachvollziehbarkeit auf Kosten einer scheinbar bequemerer automatischen Bereinigung.

## **9. Öffentliche Zeitverankerung**

Nach einem erfolgreichen Quellenlauf kann für den UTC-Tag ein Manifest erzeugt werden. Es enthält die jeweils letzten Hashes der JSONL- und SQLite-Kette, jedoch keine Nachrichtentexte und keine Bilder. Das Manifest ist klein genug, um öffentlich veröffentlicht und unabhängig geprüft zu werden. Es repräsentiert den lokalen Archivzustand nicht vollständig, sondern über seine beiden Kettenendpunkte.

OpenTimestamps versieht die Manifestdatei mit einem attestierten Zeitpfad. Die Dokumentation beschreibt einen Zeitstempel als Nachweis dafür, dass bestimmte Daten vor einem Zeitpunkt existierten, und trennt ausdrücklich das Erzeugen vom späteren unabhängigen Verifizieren [10]. Der Befehl `ots stamp` verarbeitet die Datei und erzeugt eine zugehörige `.ots`-Datei. Eine spätere Prüfung muss zunächst den lokalen Inhalt und die Kette verifizieren, dann die aktuellen Kettenwerte mit dem Manifest vergleichen und erst danach den Zeitnachweis prüfen. Nur diese Reihenfolge verhindert, dass ein gültiger Zeitstempel für einen Hash fälschlich als Beleg für einen beliebigen Inhalt interpretiert wird.

Die GitHub-Synchronisierung ergänzt die externe Zeitverankerung. Manifest und Proof-Datei werden in einem Repository versioniert veröffentlicht. GitHub liefert damit Auffindbarkeit und eine sichtbare Veröffentlichungshistorie; OpenTimestamps liefert den unabhängigen zeitlichen Bezug. Bestehende Dateien werden inhaltlich verglichen und unveränderte Dateien nicht erneut per API hochgeladen. So werden unnötige Git-Commits vermieden.

Die gewählte Konstruktion wahrt eine wichtige Datenminimierung. Der externe Dienst erhält keine Artikelinhalte. Dennoch ist ein Manifest nicht geheim: Hashwerte können unter bestimmten Umständen Informationen über bekannte Inhalte oder Zustände verraten. Die Entscheidung, welche Manifeste und welche lokalen Daten veröffentlicht werden, bleibt deshalb eine Frage des Betriebs- und Datenschutzkonzepts, nicht nur eine technische Voreinstellung.

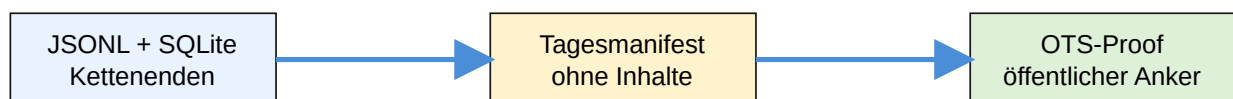


Abbildung 3: Das Manifest vermittelt zwischen lokalen Daten und externem Zeitnachweis.

## 10. WebUI und Beobachtbarkeit

Die WebUI stellt die technische Archivspur in einer für Menschen prüfbaren Form dar. Quellenkarten zeigen Anzahl der Meldungen, Bilder, Fehler und den Anchor-Status. Eine Meldungsliste unterstützt Quellfilter, Pagination und den Übergang zur Detailseite. Die Detailansicht verbindet Nachrichtentext, Zeitangaben, Hashes und gespeicherte Bilder. Damit



werden nicht nur Metriken, sondern auch die konkreten Objekte sichtbar, auf die sich eine Integritätsprüfung bezieht.

Die Anwendung verarbeitet Zeitstempel intern in UTC. Erst im Browser werden sie in die lokale Zeitzone umgerechnet. Diese Trennung verhindert, dass der Rechner des Daemons und der Rechner einer lesenden Person dieselbe Meldung mit verschiedenen gespeicherten Zeitwerten versehen. Für wissenschaftliche Auswertungen ist dennoch zu dokumentieren, ob eine Anzeigezeit oder der kanonische UTC-Wert verwendet wurde.

Prometheus-Metriken ergänzen die fachliche Oberfläche um maschinenlesbare Betriebsdaten. Erfasst werden unter anderem konfigurierte Quellen, Datensätze, Bilder und Quellenfehler. Ein optionaler Heartbeat meldet den Prozessbetrieb an einen konfigurierten Dienst. Beide Mechanismen beantworten jedoch nur Betriebsfragen. Sie zeigen, ob der Dienst läuft oder welche Zähler er führt; sie prüfen nicht selbst, ob der Inhalt einer Kette korrekt ist.

Die WebUI verzichtet auf Benutzerverwaltung und Authentifizierung, weil der lokale Betrieb im Vordergrund steht. Diese Vereinfachung ist eine klare Einsatzgrenze. Für eine öffentliche Bereitstellung wären mindestens Netzwerksegmentierung, Reverse-Proxy-Schutz, Zugriffskontrolle und eine Prüfung der schreibenden Endpunkte erforderlich. Ein Dashboard ist kein Sicherheitsmechanismus, sondern eine Beobachtungsschnittstelle.

## **11. Verifikation und Reproduzierbarkeit**

Eine unabhängige Verifikation kann in mehreren Stufen erfolgen. Zuerst wird geprüft, ob jede JSONL-Zeile syntaktisch lesbar ist und ob die SQLite-Tabellen dem erwarteten Schema entsprechen. Danach werden Datensätze in der gespeicherten Reihenfolge gelesen, die Vorgängerbeziehungen nachvollzogen und die Hashes aus dem dokumentierten Material neu berechnet. Die Ergebnisse der beiden Formate dürfen voneinander abweichen, wenn ein Speicher während eines Abbruchs voraus war; jedes Format muss aber seine eigene Kette erklären können.

Für die Prüfung eines Anchors werden anschließend Manifest und .ots-Datei benötigt. Der Prüfer vergleicht die im Manifest genannten Endhashes mit den lokal validierten Ketten. Erst wenn diese Werte übereinstimmen, besitzt der externe Zeitrachweis eine Aussage über die lokale Archivspur. Ein Proof ohne passende lokale Daten ist lediglich der Nachweis, dass ein bestimmter Hash existierte.

Reproduzierbarkeit umfasst nicht nur Softwareversionen. Feed-URL, Quellenkonfiguration, Codec, Abrufintervall, Netzwerkantwort, Browser-Version, Viewport und Zeitpunkt können die archivierte Fassung beeinflussen. JSON- und XML-Feeds lassen sich häufig strukturell reproduzieren; eine dynamische Zielseite oder ein Screenshot ist stärker kontingent. Die

Architektur sollte diese Unterschiede dokumentieren, statt sie unter einer einheitlichen Genauigkeitsbehauptung zu verbergen.

Das Projekt unterstützt Reproduzierbarkeit durch ``pyproject.toml``, ``uv.lock``, Tests und Projektdokumentation. Für einen dauerhaften wissenschaftlichen Datensatz wären zusätzlich Konfigurationssnapshots, Prüfsummen der verwendeten Software und eine unveränderliche Kopie der Rohantworten empfehlenswert. Diese Erweiterungen sind mit zusätzlichen Speicher- und Datenschutzkosten verbunden und gehören daher in eine explizite Forschungs- oder Aufbewahrungsstrategie.

## **12. Evaluation durch Szenarien**

Im Normalbetrieb wird ein neuer Feeddatensatz abgerufen, normalisiert, auf bekannte Quellenkennung geprüft, um Bilder oder Zusatzinhalte ergänzt, gehasht und in beide Speicher geschrieben. Die erwartete Kette wächst um genau ein Element pro Speicherformat. Die Dashboard-Liste zeigt den Datensatz; die Detailansicht kann Text und Bilder öffnen. Dieses Basisszenario prüft die Verbindung der Hauptkomponenten, sagt aber noch wenig über Fehlertoleranz aus.

Beim Wiederholungsabruf derselben Meldung erkennt die Deduplizierung die bekannte Quellenkennung und vermeidet einen weiteren Ketteneintrag. Wird eine neue Quellenfassung unter einer neuen Kennung geliefert, entsteht ein neuer Datensatz. Diese Regel ist einfach und effizient, verlagert aber einen Teil der Identitätssemantik in die Quelle. Eine Anwendung kann nicht allein aus einem Titel sicher ableiten, ob zwei redaktionelle Fassungen fachlich identisch sind.

Bei einem fehlerhaften Feed wird die Quelle als fehlerhaft markiert, während andere Quellen weiterlaufen. Bei einem Prozessabbruch zwischen JSONL- und SQLite-Schritt können die Ketten unterschiedliche Enden haben. Bei einer nachträglichen manuellen Änderung schlagen die Hashprüfungen fehl. Bei einer inkompatiblen Tabelle wird die alte Struktur umbenannt. Diese Szenarien zeigen verschiedene Schichten der Robustheit: Isolation, Erkennung, Erhaltung und kontrollierten Wiederanlauf.

Die Tests des Projekts decken Settings-Validierung, Feednormalisierung, Codecverhalten, Hash-Ketten, Sharding, SQLite-Schema, Anchoring, GitHub-Synchronisierung, Daemon-Polling und WebUI-Rendering ab. Screenshot-Tests laden Seiten mit `domcontentloaded` und einer kurzen Renderwartezeit statt mit `networkidle`. Damit wird die Testumgebung auch für Seiten mit dauerhaften Netzwerkverbindungen deterministischer.

## **13. Bedrohungsmodell und Datenschutz**

Das Bedrohungsmodell umfasst unbemerkte lokale Änderungen, beschädigte Dateien, unvollständige Schreibvorgänge, fehlende Quellen und eine verfälschte Eingabe vor dem Hashing. Gegen nachträgliche lokale Änderungen bietet die Kette eine starke Sichtbarkeit, sofern der Prüfer auf eine vertrauenswürdige vorherige Kette oder ein veröffentlichtes Manifest zurückgreifen kann. Gegen einen Betreiber, der von Beginn an falsche Inhalte speichert, hilft die Kette nicht. Sie kann auch nicht beweisen, dass ein nicht abgerufener Artikel nie existiert hat.

Ein vollständiges Löschen des lokalen Archivs wird durch eine lokale Hash-Kette nicht verhindert. Dafür sind unabhängige Backups oder öffentlich veröffentlichte Nachweise erforderlich. Ein tägliches Manifest schützt außerdem nur den darin repräsentierten Kettenendpunkt. Die Granularität des Nachweises ist daher geringer als die Granularität einzelner Nachrichten. Diese Eigenschaft ist ein bewusster Kompromiss zwischen Datenschutz, Kosten und Prüfwert.

Nachrichten können personenbezogene Daten, sensible Aussagen oder urheberrechtlich geschützte Inhalte enthalten. Die Tatsache, dass ein Inhalt technisch gehasht werden kann, begründet kein Recht zu seiner dauerhaften Speicherung. Quellenwahl, Aufbewahrungsdauer, Zugriffsrechte und Löschverfahren müssen rechtlich und redaktionell bewertet werden. OpenTimestamps reduziert zwar den externen Inhaltsabfluss, macht aber die lokale Verantwortung nicht überflüssig.

Auch die WebUI ist Teil der Bedrohungsfläche. Ohne Authentifizierung dürfen Abruf-, Medien- und Shutdown-Endpunkte nicht öffentlich erreichbar sein. Credentials für GitHub werden aus einer lokalen Datei gelesen und nicht geloggt; sichere Dateirechte und Secret-Verwaltung obliegen der Umgebung. Der Artikel versteht diese Punkte als Systemanforderungen und nicht als optionale Komfortmaßnahmen.

## **14. Grenzen der Aussagekraft**

Die zentrale Aussage der Hash-Kette ist eng: Die gespeicherte Repräsentation entspricht der erwarteten Kette oder eine Abweichung lässt sich erkennen. Daraus folgt nicht, dass ein Artikel wahr, ausgewogen, vollständig oder frei von Fehlern ist. Ein lokales Archiv kann eine irreführende Quelle sehr zuverlässig unverändert bewahren. Integrität und Qualität sind unterschiedliche Eigenschaften und müssen in einer wissenschaftlichen Auswertung getrennt berichtet werden.

Auch die zeitliche Aussage ist begrenzt. Ein OpenTimestamps-Proof belegt die Existenz eines Hashes spätestens zu einem attestierten Zeitpunkt. Er beweist nicht, dass der darin referenzierte Nachrichteninhalte damals bereits öffentlich war, sofern keine weitere Quelle diese Verbindung

dokumentiert. Der Proof ist ein Existenznachweis für eine Bytefolge, keine unabhängige redaktionelle Beglaubigung.

Die Codecarchitektur erhöht die Abdeckung, kann aber auch neue Unsicherheiten erzeugen. Der vollständige Abruf eines Artikels hängt von dessen Verfügbarkeit und der Interpretation durch den Codec ab. Ein Screenshot hängt von Browser, Bildschirmgröße, dynamischen Ressourcen und Consent-Zuständen ab. Deshalb muss die jeweils verwendete Methode zusammen mit dem Archivdatensatz ausgewertet werden. Ein Textrecord und ein visueller Snapshot sind verschiedene Beobachtungsobjekte.

Schließlich ist die Anwendung als lokaler Dienst keine globale Infrastruktur. Der Betrieb kann ausfallen, Quellen können sich verändern und eine einzelne Instanz kann nicht alle Perspektiven des Nachrichtenraums abbilden. Der Wert des Projekts liegt nicht in einer totalen Repräsentation, sondern in der transparenten Dokumentation eines begrenzten, technisch prüfbaren Ausschnitts.

## **15. Weiterentwicklung**

Eine erste Weiterentwicklung wäre die explizite Versionierung von Schema, Hashalgorithmus und Codec. Damit könnten mehrere Kettengenerationen nebeneinander geprüft werden, ohne dass eine spätere Änderung der Kanonisierung als Datenkorruption erscheint. Ergänzend wäre ein eigenständiges Verifikationswerkzeug sinnvoll, das JSONL, SQLite, Manifest und Proof lesen kann, ohne den laufenden Daemon und seine Netzwerkabhängigkeiten zu starten.

Für die Langzeitarchivierung könnten unveränderliche oder geografisch getrennte Replikate eingesetzt werden. Ein Replikationsprotokoll müsste dabei nicht nur Dateien kopieren, sondern auch den Zeitpunkt und die Integritätsprüfung jeder Kopie dokumentieren. Die öffentliche GitHub-Synchronisierung ist für kleine Manifeste geeignet, ersetzt aber keine belastbare Sicherungsstrategie für große Bild- und Nachrichtendaten.

Die Beobachtbarkeit könnte um Soll-Ist-Vergleiche für Abrufintervalle, fehlende Meldungen und ungewöhnliche Größenänderungen erweitert werden. Für Screenshots könnten standardisierte Viewports, Browser-Versionen, Ladezeiten und eine Liste geladener Ressourcen gespeichert werden. Dadurch würde die visuelle Archivierung besser vergleichbar, aber auch stärker an konkrete Softwareumgebungen gebunden.

Eine weitere Forschungsrichtung wäre die Untersuchung von Quellenvielfalt und zeitlicher Verzerrung. Das technische Archiv kann mehrere Quellen parallel sichtbar machen, bewertet deren journalistische Qualität aber nicht. Metadaten über Quelle, Codec und Abrufhäufigkeit könnten für eine spätere Medienanalyse genutzt werden, sofern sie nicht mit einer automatischen Objektivitätsbehauptung verwechselt werden.

## **16. Anforderungen als überprüfbare Invarianten**

Ein technisches Archiv wird belastbarer, wenn seine Anforderungen nicht nur als Wunschliste, sondern als Invarianten formuliert werden. Für News-Hash lautet eine erste Invariante, dass ein bereits gespeicherter Datensatz nicht durch einen späteren Normalbetrieb überschrieben oder gelöscht wird. Eine zweite Invariante fordert, dass jeder Datensatz in jeder Speicherform eine nachvollziehbare Vorgängerbeziehung besitzt. Eine dritte verlangt, dass bekannte Quellenkennungen vor teuren Verarbeitungsschritten erkannt werden. Diese Regeln sind operationalisierbar und können deshalb durch Tests und Prüfwerkzeuge beobachtet werden.

Weitere Anforderungen betreffen den Betrieb. Ein Fehler einer einzelnen Quelle darf die übrigen Quellen nicht stoppen. Der Daemon muss individuelle Abrufintervalle respektieren. Laufzeitfehler sollen sichtbar werden, ohne in den unveränderlichen Nachrichtenbestand einzufließen. Die WebUI muss Quellenfilter, Pagination, Detailansichten und gespeicherte Bilder anbieten. Ein tägliches Manifest muss die Kettenendpunkte so zusammenfassen, dass ein externer Dienst keine Nachrichtentexte benötigt.

Die Unterscheidung zwischen Muss-, Soll- und Nicht-Scope-Anforderungen verhindert Überdehnung. Das Projekt muss JSON- und XML-Feeds verarbeiten, Hash-Ketten führen und OpenTimestamps-Proofs erzeugen können. Bilder, lokale Zeitzonen und flüchtige Fehlerzähler sind wichtige Soll-Funktionen. Eine öffentliche Benutzerverwaltung, eine eigene Blockchain und das Speichern von Nachrichtentexten auf einer öffentlichen Blockchain liegen dagegen außerhalb des bewusst gewählten Umfangs.

Invarianten helfen auch bei der Kommunikation mit späteren Nutzerinnen und Nutzern. Statt allgemein von „Sicherheit“ zu sprechen, kann dokumentiert werden, welche Eigenschaft geprüft wird: Kettenintegrität, Schemaverträglichkeit, Deduplizierung oder externe zeitliche Existenz. Diese Präzision senkt das Risiko, dass eine technisch korrekte Funktion mit einem weitergehenden gesellschaftlichen Versprechen verwechselt wird.

## **17. Fachliches Datenmodell**

Der zentrale Datensatz bildet eine normalisierte Nachricht ab. Er enthält eine Quellenkennung, Titel, Quell-URL, Autor, Inhalt, Veröffentlichungszeit, Codecname, Bildmetadaten und die Hashbeziehung. Diese Felder sind fachlich voneinander zu unterscheiden. Die Quellenkennung dient der Wiedererkennung, die URL verweist auf die beobachtete Ressource, der Inhalt beschreibt die archivierte Repräsentation und der Veröffentlichungszeitpunkt ordnet sie zeitlich ein. Der technische Abrufzeitpunkt gehört zum Betriebsereignis, aber nicht zum unveränderlichen Inhaltskern.

Die Trennung von Veröffentlichungs- und Abrufzeitpunkt ist für spätere Analysen wesentlich. Eine Meldung kann am Montag veröffentlicht und am Dienstag erstmals vom Archiv erreicht worden sein. Werden beide Zeitpunkte verwechselt, entsteht der falsche Eindruck einer verspäteten Veröffentlichung. News-Hash speichert beide Werte, verwendet aber nur den fachlich relevanten Veröffentlichungszeitpunkt im Hashmaterial. In der WebUI wird der Zeitpunkt der Anzeige zusätzlich an die lokale Browserzeitzone angepasst.

Bilder werden als strukturierte Metadaten und als Binärdaten behandelt. Ein Bild-Hash kann der Deduplizierung dienen, während Dateiname, Quelle und Veröffentlichungszeit die Rekonstruktion unterstützen. JSONL verweist auf eine relative Datei. SQLite hält das Bild zusätzlich als BLOB, damit eine Detailansicht nicht von einer einzelnen JSONL-Datei abhängig ist. Diese doppelte Repräsentation vergrößert den Speicherbedarf, erhöht aber die Robustheit gegenüber partiellen Dateiproblemen.

Das Datenmodell ist absichtlich nicht als vollständiges journalistisches Ontologiesystem angelegt. Es beschreibt, was der Importprozess beobachten und speichern kann. Redaktionelle Rollen, Korrekturgründe, Lizenzbedingungen oder semantische Beziehungen zwischen Artikeln wären wertvolle Erweiterungen, müssen aber mit eigener Terminologie und eigener Provenienzlogik eingeführt werden. Ein kleines stabiles Modell ist für einen ersten Integritätsdienst oft besser prüfbar als ein umfassendes, aber unklar verwendetes Schema.

## **18. Kanonische Serialisierung und Determinismus**

Hashverfahren sind nur dann aussagekräftig, wenn dieselben fachlichen Werte in derselben Bytefolge verarbeitet werden. JSON erlaubt jedoch unterschiedliche Schlüsselreihenfolgen und optionale Leerzeichen. Deshalb baut News-Hash ein Hashobjekt auf, sortiert die Schlüssel und serialisiert ohne zusätzliche Leerzeichen. Die anschließende UTF-8-Kodierung bildet die konkrete Eingabe für SHA-256. Der Determinismus liegt damit nicht in JSON allgemein, sondern in einer festgelegten Variante der JSON-Verwendung.

Auch die Behandlung fehlender und leerer Werte muss stabil sein. Ein fehlendes Autorenfeld darf nicht zufällig einmal als leerer String und einmal als nicht vorhandener Schlüssel erscheinen, wenn beide Zustände fachlich unterschiedlich bewertet werden. Gleiches gilt für Bilder, Datumsformate und HTML-Inhalte. Die Normalisierung ist deshalb eine semantische Vorstufe der Kryptografie. Der Hash kann nur so zuverlässig sein wie die Entscheidung, welche Unterschiede bedeutungstragend sind.

Zeitzone sind ein besonders anschauliches Beispiel. Zwei Datumsangaben können denselben Zeitpunkt ausdrücken, aber textuell verschieden aussehen. Werden Rohstrings gehasht, erzeugen sie möglicherweise unterschiedliche Hashes. Eine UTC-Normalisierung vor der Serialisierung

reduziert diese künstliche Varianz. Gleichzeitig darf die Normalisierung keine Information entfernen, die für eine spätere Quellenkritik benötigt wird. Deshalb können Originalwerte in ergänzenden Feldern erhalten bleiben, während der kanonische Wert für die Kette bestimmt wird.

Für zukünftige Kettenversionen wäre ein explizites Kanonisierungsprofil sinnvoll. Es könnte Feldnamen, Zeichensatz, Datumsregeln, Bildreferenzen und Algorithmus in einer maschinenlesbaren Beschreibung festhalten. Ein Prüfer müsste dann nicht aus dem Quellcode erschließen, welche Serialisierung im jeweiligen Zeitraum galt. Diese Erweiterung würde Migration und Langzeitprüfung erleichtern, ist aber selbst eine neue Spezifikation, die versioniert und getestet werden müsste.

## **19. Speicherorganisation und Sharding**

Ein dauerhaft laufender Nachrichtenimport erzeugt unweigerlich große Dateien. Die Shard-Organisation teilt den Bestand pro Quelle und Speicherformat in nummerierte Einheiten. Der Namensbestandteil `storage_name` beschreibt die Quelle, die Nummer den laufenden Shard. Die Trennung unterstützt Backups, begrenzt die maximale Dateigröße und erlaubt eine schrittweise Archivierung. Sie stellt aber auch Anforderungen an Abfragen: Eine historische Suche muss mehrere Shards zusammenführen, während der häufige Import nicht jedes Mal den gesamten Bestand lesen sollte.

News-Hash entscheidet sich deshalb für unterschiedliche Lesestrategien. Für Deduplizierung und Dashboard-Vorschau wird bevorzugt der neueste Shard genutzt. Kennzahlen und Detailzugriffe können über alle Shards aggregieren. Diese Asymmetrie ist nicht unsichtbar, sondern Teil der Systemdokumentation. Ein Nutzer muss wissen, ob eine Anzeige den vollständigen Archivbestand oder nur den aktuellen Arbeitsbestand repräsentiert.

Shard-Grenzen dürfen keine semantischen Grenzen erzeugen. Der Hash des ersten Datensatzes eines neuen Shards muss auf den letzten Datensatz des vorherigen Shards verweisen, wenn beide zur selben Speicherketten gehören. Andernfalls würde die Kette an jeder Dateigrenze künstlich neu beginnen. Die Dateiteilung ist daher eine physische Organisation, während die Hashbeziehung eine logische Kontinuität bewahrt.

Für eine große Installation wären zusätzlich Indexdateien oder ein Manifest pro Shard denkbar. Solche Metadaten könnten minimale und maximale Zeitpunkte, Datensatzanzahl, ersten und letzten Hash sowie Prüfergebnisse enthalten. Sie würden schnelle Übersichten ermöglichen, müssten aber selbst gegen nachträgliche Veränderung geschützt werden. Jede Optimierung der Abfrageführung erweitert damit die Menge der Zustände, die eine Archivprüfung berücksichtigen muss.

## **20. Codec-Schnittstelle als Erweiterungspunkt**

Ein allgemeiner Feedstandard beschreibt selten alle Informationen, die für eine Archivierung relevant sind. Manche Quellen liefern nur kurze Anreißer, andere stellen Bilder über relative URLs bereit oder nutzen proprietäre Metadatenfelder. Die Codec-Schnittstelle isoliert diese Unterschiede. Ein Codec erhält die externe Darstellung, ruft bei Bedarf ergänzende Ressourcen ab und gibt einen normalisierten Datensatz zurück. Die Hash- und Speicherlogik arbeitet anschließend mit dem gemeinsamen Ergebnis.

Die Erweiterung über Codecs ist ein Beispiel für eine Architekturentscheidung zugunsten lokaler Verständlichkeit. Ein neuer Anbieter muss nicht in einer großen Fallunterscheidung des Daemons ergänzt werden. Stattdessen kann die Konfiguration auf einen Codec verweisen, der seine eigenen Annahmen kapselt. Diese Modularität erleichtert Tests: Ein Codec kann mit repräsentativen Eingaben geprüft werden, ohne einen vollständigen Dauerbetrieb zu starten.

Die Kehrseite ist die Verantwortung für zusätzliche Netzwerkanfragen. Der TAZ-Codec kann an einem Feedlink einen vollständigen Artikel abrufen; der Screenshot-Codec startet einen Browser. Jede zusätzliche Anfrage kann fehlschlagen, länger dauern oder andere Inhalte zurückgeben als der ursprüngliche Feed. Der resultierende Datensatz sollte deshalb nachvollziehbar machen, welche Codecstrategie ihn erzeugt hat. Der Codecname ist nicht bloß ein internes Implementierungsdetail, sondern Teil der Provenienz.

Eine stabile Codec-Schnittstelle muss zudem mit partiellen Ergebnissen umgehen. Wenn der Feed vorhanden, der Artikeltext aber nicht erreichbar ist, kann die Anwendung entweder den Feeddatensatz speichern, den gesamten Datensatz verwerfen oder einen expliziten Fehlerzustand erzeugen. Die richtige Entscheidung hängt vom Archivziel ab. News-Hash behandelt solche Fälle als quellspezifische Betriebs- und Datenqualitätssituationen, die nicht stillschweigend mit einer vollständigen Artikelarchivierung gleichgesetzt werden dürfen.

## **21. Visuelle Archivierung mit SCREENv0**

Ein Screenshot ist eine besondere Form der Quellenaufnahme. Textnormalisierung versucht, semantische Inhalte aus einer Darstellung zu extrahieren. Ein Screenshot bewahrt dagegen die Darstellung selbst: Typografie, Anordnung, sichtbare Bilder und den Zustand interaktiver Elemente zum Zeitpunkt der Aufnahme. Für bestimmte Untersuchungen, etwa die spätere Analyse von Überschriften oder Seitengestaltung, ist diese visuelle Information unverzichtbar.

Die visuelle Methode führt jedoch mehr Einflussfaktoren ein. Browserengine, Fenstergröße, Geräteskalierung, Schriftverfügbarkeit, Ladezeit und Netzwerkantwort können das Ergebnis verändern. News-Hash verwendet Chromium und eine kontrollierte Renderwartezeit. Bekannte Cookie- und Consent-Banner werden entfernt, damit ein temporäres Overlay nicht den



eigentlichen Seiteninhalt verdeckt. Diese Regel ist praktisch, aber nicht universell: Neue Seiten können neue Banner oder Zustimmungselemente verwenden.

Ein wissenschaftlicher Prüfer sollte einen Screenshot daher als Beobachtung unter Bedingungen lesen. Das Bild beantwortet nicht, wie die Seite auf einem anderen Gerät ausgesehen hätte. Es beweist auch nicht, dass alle dynamischen Inhalte geladen waren. Eine bessere Dokumentation könnte Viewport, Browser-Version, Ladezeit, Screenshot-Hash und geladene Ressourcen zusammen speichern. Dadurch würde die spätere Vergleichbarkeit wachsen, aber auch die technische Beschreibung jedes Bildrecords komplexer.

Die Methode zeigt exemplarisch, dass digitale Archivierung immer eine Auswahl ist. Die Entscheidung, ein PNG zu speichern, legt fest, welche Aspekte einer Webressource für die spätere Nutzung priorisiert werden. News-Hash verbindet deshalb Textdatensätze und Bilder, ohne sie als identische Repräsentationen auszugeben. Beide Artefakttypen gehören zur Provenienz eines Abrufs, liefern aber unterschiedliche Evidenz.

## **22. Transaktionsgrenzen und Wiederherstellung**

JSONL und SQLite können nicht ohne Weiteres in eine gemeinsame atomare Transaktion eingeschlossen werden. Ein Betriebssystemabsturz, ein voller Datenträger oder ein Prozessfehler kann nach dem Schreiben in den ersten, aber vor dem Schreiben in den zweiten Speicher auftreten. Das System muss diesen Fall nicht verbergen, sondern so gestalten, dass der verbleibende Zustand erkennbar und wiederaufnehmbar ist.

Die getrennten Hash-Ketten sind hierfür eine wichtige Beobachtungseinrichtung. Wenn der JSONL-Shard voraus ist, kann seine letzte Kette unabhängig validiert werden. SQLite bleibt auf seinem eigenen letzten gültigen Hash. Beim nächsten Lauf darf der Import nicht blind den längeren Zustand als globale Wahrheit annehmen; er muss pro Format feststellen, welche Quellenkennungen und welcher Vorgängerwert gelten. Die Wiederherstellung wird dadurch konservativ und datenorientiert.

Eine alternative Architektur könnte zunächst in eine transaktionale Zwischenablage schreiben und daraus beide Formate erzeugen. Das würde die gemeinsame Sicht vereinfachen, verschiebt aber den Fehlerpunkt und erhöht die Zahl der zu verwaltenden Zustände. Außerdem würde JSONL seine Rolle als direkte, append-only Archivspur verlieren. News-Hash bevorzugt die einfache Sichtbarkeit zweier unabhängig prüfbarer Ergebnisse gegenüber einer komplexeren vermeintlichen Gesamtatomizität.

Für eine produktive Wiederherstellungsprozedur wären Prüfschritte zu dokumentieren: Prozess stoppen, letzte gültige Hashes je Format ermitteln, unvollständige Zeilen erkennen, SQLite-

Schema prüfen, Datenbestand sichern und erst danach den Daemon neu starten. Automatisierte Reparatur sollte immer eine neue protokollierte Operation sein. Eine stillschweigende Korrektur würde genau die historische Nachvollziehbarkeit beeinträchtigen, die das System herstellen soll.

### **23. Manifestbildung und unabhängige Prüfung**

Das Manifest ist eine bewusste Verdichtung. Die lokalen Speicher können Millionen von Zeichen und viele Bilder enthalten, während das Manifest nur wenige Endhashes und Metadaten benötigt. Seine Rolle ähnelt damit einem Inhaltsverzeichnis für Integrität, nicht einem Inhaltersatz. Der Prüfer benötigt weiterhin Zugriff auf die lokale Kette, wenn er den Hash selbst aus den Nachrichtendaten neu berechnen möchte.

Ein robuster Prüfablauf sollte drei Ebenen unterscheiden. Auf der ersten Ebene wird die syntaktische Lesbarkeit geprüft: Sind JSONL-Zeilen gültig, sind Bildpfade vorhanden und entspricht SQLite dem erwarteten Schema? Auf der zweiten Ebene wird die Kette geprüft: Stimmen Vorgängerwerte und neu berechnete Hashes? Auf der dritten Ebene wird das Manifest verglichen und der OpenTimestamps-Proof verifiziert. Ein Fehler auf einer unteren Ebene macht eine höhere Bestätigung nicht zu einem Beleg für korrekte Inhalte.

Die tägliche Granularität ist ein Kompromiss. Ein einzelner Manifestwert kann viele Meldungen indirekt repräsentieren, ist aber weniger aussagekräftig als ein Proof pro Datensatz. Ein Proof pro Meldung wäre teurer, schwerer zu verwalten und würde womöglich mehr Metadaten veröffentlichen. Die Wahl des Tages als Einheit passt zum Betrieb des Daemons und zur öffentlichen GitHub-Synchronisierung.

Für spätere Nutzer sollte ein Manifest mit Quelle, UTC-Datum, Speicherformaten, Hashalgorithmus und Kettenversion verständlich beschriftet sein. Technische Kürze darf nicht zu semantischer Unklarheit führen. Ein Prüfer muss wissen, ob ein Hash den gesamten Datensatz, nur einen Inhaltsteil oder ein Kettenende repräsentiert. Die Dokumentation ist damit ein Teil der Beweiskette und nicht bloß Begleittext.

### **24. Teststrategie und Qualitätssicherung**

Die Qualitätssicherung folgt der Schichtung der Anwendung. Settings-Tests prüfen, ob Quellenblöcke vollständig und plausibel sind. Codec-Tests prüfen Feldzuordnung, Zeitnormalisierung, Bildbehandlung und Sonderfälle. Hash-Tests prüfen kanonische Serialisierung, Vorgängerbeziehungen und Erkennung von Änderungen. Storage-Tests prüfen Sharding, Schemawechsel, Deduplizierung und parallele Speicherung. Webtests prüfen gerenderte HTML-Strukturen, Endpunkte und Dashboarddaten.

Diese Testarten haben unterschiedliche Aussagekraft. Ein Unit-Test kann zeigen, dass eine Funktion bei einem bekannten Beispiel korrekt reagiert. Er beweist nicht, dass eine externe Website dieselbe Struktur beibehält. Ein Integrationstest kann den Übergang zwischen Feed, Codec und Storage prüfen, aber nicht die gesamte Lebensdauer eines Archivs. Die Testabdeckung sollte daher nach Risiken und nicht nur nach Zeilenzahl beurteilt werden.

Browser- und Screenshottests benötigen eine kontrollierte Umgebung. Dauerhafte Verbindungen können dazu führen, dass `networkidle` nie erreicht wird. Die Verwendung von `domcontentloaded` mit einer kurzen Renderwartezeit bildet den praktischen Zweck besser ab. Vor der Aufnahme werden bekannte Banner entfernt. Ein neuer Codec sollte deshalb nicht nur funktional getestet, sondern auch visuell gegen die vorgesehenen Archivbedingungen geprüft werden.

Qualitätssicherung umfasst außerdem die Dokumentation. Architektur, Anforderungen, Vision, WebUI und Nutzerhilfe müssen dieselbe technische Realität beschreiben. Eine nicht aktualisierte Dokumentation kann bei einem späteren Prüfer einen Widerspruch erzeugen, obwohl der Code intern konsistent ist. In einem Integritätsprojekt ist diese kommunikative Konsistenz besonders relevant, weil der Prüfer nicht nur Dateien, sondern auch deren Bedeutung rekonstruieren muss.

## **25. Deployment und Betriebspraxis**

News-Hash kann direkt mit Python und `uv` oder in einem Docker-Container betrieben werden. Die Paketmetadaten und das Lockfile halten die Abhängigkeiten reproduzierbar. Für `SCREENv0` muss Chromium installiert sein. Die Laufzeitdaten liegen getrennt vom Anwendungscode unter `app/data/`; dort werden Einstellungen, JSONL- und SQLite-Dateien, Bilder, Logs und Anchor-Dateien verwaltet.

Ein sicherer Betrieb beginnt mit einer klaren Verzeichnis- und Berechtigungsstrategie. Credentials für GitHub gehören in eine lokale Umgebungsdatei, die nicht in Logs oder öffentliche Repositories gelangt. Schreibrechte auf Archivdateien sollten nur dem Dienstkonto gewährt werden. Backups müssen während oder nach einer Integritätsprüfung erstellt werden, damit nicht gerade ein unvollständiger Zustand als verlässliche Kopie verteilt wird.

Die Überwachung sollte Abrufintervalle, Datenwachstum, freie Speicherkapazität, Fehler pro Quelle und den Zustand des heutigen Anchors beobachten. Ein Heartbeat kann einen abgestürzten Prozess anzeigen, aber nicht erkennen, dass eine Quelle seit Tagen leer antwortet. Dafür sind fachliche Prüfungen nötig, etwa ein Vergleich erwarteter Aktualisierungsraten oder eine Warnung bei ungewöhnlich langen Lücken.

Ein Container vereinfacht die Bereitstellung von Python und Chromium, löst aber keine Datenhaltungsfragen. Das Volume für `/app/data` muss gesichert und auf ausreichende Größe

geprüft werden. Ein neu erzeugter Container darf nicht als Ersatz für ein Backup verstanden werden. Für die Langzeitarchivierung sind die Daten, die Manifeste, die Proofs und die Beschreibung ihrer Entstehungsbedingungen gemeinsam aufzubewahren.

## **26. Quellenvielfalt und Medienvergleich**

Ein Archiv, das mehrere Quellen parallel abruft, kann zeitliche und redaktionelle Unterschiede sichtbar machen. Titel können variieren, Veröffentlichungszeiten auseinanderliegen und Themen unterschiedlich lange präsent bleiben. News-Hash erzeugt daraus jedoch keine automatische Medienanalyse. Es bewahrt die Eingangsdaten in einer Form, die spätere Vergleiche ermöglicht, ohne eine redaktionelle Bewertung in die technische Speicherung einzubauen.

Für einen quellenübergreifenden Vergleich müssen Identitätsprobleme gelöst werden. Zwei Quellen können dasselbe Ereignis mit unterschiedlichen IDs, Titeln und Zeitangaben beschreiben. Eine reine Quellenkennung reicht dann nicht für eine gemeinsame Ereignisgruppe. Eine spätere Analyseschicht könnte Ähnlichkeit, Links oder manuelle Annotationen verwenden. Solche Ableitungen sollten als neue Forschungsergebnisse gekennzeichnet werden und nicht mit den ursprünglichen Archivrecords verschmelzen.

Auch die Auswahl der Quellen beeinflusst die Aussagekraft. Ein Archiv mit deutschsprachigen RSS-Feeds zeigt nicht den gesamten Nachrichtenraum. Ein Screenshot-Codec kann bestimmte technisch erreichbare Medien bevorzugen, während Paywalls oder robots.txt andere Quellen ausschließen. Transparenz über Konfiguration und Ausfälle ist deshalb eine Voraussetzung für jede gesellschaftliche Interpretation. Die Hash-Kette macht eine Auswahl nicht neutral.

Die parallele Sichtbarkeit verschiedener Quellen hat dennoch einen praktischen Wert. Sie ermöglicht Rückfragen wie: Wann tauchte ein Thema erstmals auf? Wie änderte sich die Überschrift? Welche Bilder wurden verwendet? Welche Quelle blieb dauerhaft erreichbar? Solche Fragen können mit einem unveränderten Archivbestand besser untersucht werden als mit einer Sammlung aktueller Links.

## **27. Datenschutz, Recht und verantwortliche Archivierung**

Die technische Möglichkeit, Nachrichten lokal zu speichern, ist keine abschließende Antwort auf Datenschutz- und Urheberrechtsfragen. Artikel können Namen, Kontaktdaten, Gesundheitsinformationen oder andere sensible Angaben enthalten. Bilder können Personen zeigen. Die Entscheidung für eine Quelle und eine Aufbewahrungsdauer muss deshalb die rechtlichen Rahmenbedingungen des konkreten Betriebs berücksichtigen. Ein lokales Archiv reduziert externe Übertragung, beseitigt aber nicht die Verantwortung für den lokalen Datenbestand.

Besonders anspruchsvoll ist die Spannung zwischen Unveränderlichkeit und berechtigten Löscher oder Korrekturinteressen. Eine append-only Kette ist für historische Nachvollziehbarkeit wertvoll, kann aber mit einer späteren Pflicht zur Entfernung kollidieren. Technische Lösungen könnten Verschlüsselung, Zugriffsbeschränkung, getrennte Schlüsselverwaltung oder dokumentierte Löschmarkierungen vorsehen. Keine davon darf stillschweigend die Integritätsaussage einer früheren Archivfassung verfälschen.

Die Veröffentlichung von Manifesten ist datenärmer als die Veröffentlichung von Nachrichtentexten, aber nicht vollständig folgenlos. Ein Hash kann mit bekannten Dateien verglichen werden. Bei kleinen oder stark standardisierten Dateien kann dadurch eine indirekte Bestätigung entstehen. Die Entscheidung, welche Manifestinformationen öffentlich werden, sollte daher Risiko, Prüfwert und Zweck abwägen.

Verantwortliche Archivierung verlangt schließlich redaktionelle Transparenz. Nutzer sollten erkennen können, welche Quellen erfasst werden, wann Ausfälle auftreten und ob die Darstellung vollständig oder nur ausschnittshaft ist. Ein technisch elegantes System kann gesellschaftlich irreführend wirken, wenn es seine Auswahlkriterien nicht offenlegt. Dokumentation ist deshalb Bestandteil der Ethik des Projekts.

## **28. Motto und Entstehungskontext**

Das Motto „Mit KI gebaut, mit Liebe verfeinert, für neugierige Menschen“ beschreibt den Entstehungskontext des Projekts in verdichteter Form. Es ist kein wissenschaftliches Ergebnis und ersetzt keine Quellenangabe. Es benennt vielmehr drei Aspekte: die intensive Nutzung generativer Werkzeuge bei der Konstruktion, die notwendige menschliche Prüfung und die Orientierung an Personen, die Nachrichten genauer betrachten möchten.

Die Offenlegung der KI-Beteiligung ist für diesen Artikel besonders wichtig. Sowohl die News-Hash-App als auch der Artikel wurden fast vollständig KI-generiert. Menschen haben Ziel, Anforderungen, Anweisungen, Auswahl, Prüfung und Freigabe bestimmt. Die Aussage „KI-generiert“ bedeutet daher nicht, dass ein autonomes System ohne menschliche Entscheidung gehandelt hätte. Sie bezeichnet den hohen Anteil generativer Unterstützung an Code, Text, Struktur und Formulierung.

Diese Entstehungsweise verändert die Verantwortung nicht. Generierte technische Aussagen müssen gegen Quellcode, Tests und Primärquellen geprüft werden. Generierte wissenschaftliche Prosa muss auf Übertreibung, unbelegte Behauptungen und fehlerhafte Literaturangaben untersucht werden. Ein KI-Werkzeug kann eine plausible Erklärung formulieren, ohne deren Gültigkeit selbst zu garantieren. Die Transparenzmarke macht den Prüfbedarf sichtbar, statt ihn zu verbergen.

„Für neugierige Menschen“ ist dabei eine Nutzungsorientierung. Das Dashboard soll nicht nur einen technischen Hashwert zeigen, sondern Fragen ermöglichen: Was wurde wann gespeichert? Welche Quelle war beteiligt? Welche Fassung liegt vor? Was kann geprüft werden und was bleibt offen? Neugier ist in diesem Sinn keine bloße Gestaltungsidee, sondern eine Aufforderung zur methodischen Quellenkritik.

## **29. Forschungsagenda**

Aus dem Systementwurf ergeben sich mehrere offene Forschungsfelder. Ein erstes betrifft die Messung von Archivqualität. Neben Integritätsfehlern könnten Abrufvollständigkeit, zeitliche Latenz, Quellenabdeckung, Bildverfügbarkeit und Wiederherstellungsdauer als Kennzahlen untersucht werden. Solche Kennzahlen sollten nicht zu einer einzigen Qualitätszahl verschmolzen werden, weil unterschiedliche Dimensionen unterschiedliche Zielkonflikte darstellen.

Ein zweites Feld ist die formale Modellierung der Herkunft. Das W3C-PROV-Modell bietet Begriffe für Entitäten, Aktivitäten und Akteure. Eine News-Hash-Erweiterung könnte den Feedabruf, Codeschritt, Speicherakt und Manifestbildung explizit als Provenienzgraphen darstellen. Dadurch würde nachvollziehbar, welche Transformation zu welchem Datensatz geführt hat. Die zusätzliche Ausdruckskraft müsste gegen Speicherbedarf und Verständlichkeit abgewogen werden.

Ein drittes Feld betrifft verteilte Kopien ohne eine vollständige Blockchain. Mehrere unabhängige Archive könnten ihre Kettenendpunkte oder Tagesmanifeste austauschen und gegenseitig attestieren. Ein solches Netzwerk würde die Abhängigkeit von einem einzelnen Betreiber reduzieren, führte aber neue Fragen nach Teilnehmeridentität, Konfliktauflösung, Datenschutz und Verfügbarkeit ein. Die gegenwärtige lokale Architektur bildet eine kontrollierte Ausgangsbasis für solche Untersuchungen.

Ein viertes Feld ist die Benutzerforschung. Es wäre zu untersuchen, ob Nutzerinnen und Nutzer Hashes, Anchor-Status und Quellenunterschiede korrekt interpretieren. Eine Oberfläche kann technisch richtige Informationen anzeigen und dennoch falsche Sicherheit erzeugen, wenn Begriffe nicht verstanden werden. Erklärtexte, Visualisierungen und Verifikationsassistenten sollten deshalb mit realen Nutzungsszenarien evaluiert werden.

## **30. Schlussbetrachtung des erweiterten Entwurfs**

Die ausführliche Betrachtung zeigt News-Hash als Zusammenspiel vieler kleiner, nachvollziehbarer Entscheidungen. Ein Feed wird abgerufen, ein Codec normalisiert ihn, eine Quellenkennung verhindert unnötige Duplikate, ein kanonisches Objekt wird gehasht, zwei

Speicher halten unabhängige Ketten, ein Manifest verdichtet deren Endpunkte und ein Proof verankert diese Verdichtung zeitlich. Jede Stufe besitzt eigene Vorteile, Fehler und Prüfmöglichkeiten.

Der Entwurf gewinnt seine Stärke nicht durch die Behauptung vollständiger Sicherheit. Er gewinnt sie durch die Begrenzung seiner Aussagen und die Lesbarkeit seiner Prozesse. Ein Prüfer kann nachvollziehen, welche Daten gespeichert wurden, wie die Hashes entstanden, warum JSONL und SQLite unterschiedliche Enden haben können und weshalb ein OpenTimestamps-Proof keinen Nachrichteninhalt beweist. Diese Genauigkeit ist für wissenschaftliche und journalistische Nutzung wichtiger als ein größerer, aber unklarer Sicherheitsbegriff.

Die Verbindung von KI-gestützter Entwicklung, lokaler Archivierung und öffentlicher Zeitverankerung erzeugt zugleich eine interessante Metaebene. Das Projekt dokumentiert Nachrichten, wird aber selbst durch generative Verfahren hergestellt. Der Transparenzhinweis und das Motto machen diese Doppelrolle sichtbar. Auch der Artikel ist ein Artefakt, dessen Entstehung, Quellen und Grenzen nachvollziehbar bleiben müssen.

News-Hash ist damit weder neutrales Gedächtnis noch fertige Forschungsinfrastruktur. Es ist ein überprüfbarer Anfang: ein lokales Werkzeug für Menschen, die Nachrichten nicht nur lesen, sondern ihre zeitliche und technische Geschichte verstehen möchten.

### **31. Fazit**

News-Hash zeigt einen möglichen Mittelweg zwischen flüchtiger Webbeobachtung und aufwendiger verteilter Archivierungsinfrastruktur. Ein lokaler Prozess ruft konfigurierte Quellen ab, normalisiert ihre Daten und legt sie in zwei praktisch unterschiedlichen Formaten ab. Hash-Ketten machen nachträgliche Änderungen an der lokalen Spur sichtbar. Ein tägliches Manifest und OpenTimestamps erweitern diese interne Prüfbarkeit um eine öffentliche zeitliche Referenz.

Die technische Konstruktion ist deshalb überzeugend, weil sie ihre Grenzen offenlegt. Sie verspricht keine Wahrheit, keine Vollständigkeit und keine automatische rechtliche Zulässigkeit. Sie verspricht eine präzise kleinere Aussage: Eine konkrete archivierte Fassung kann gegen ihre Kette und, über ein Manifest, gegen einen externen Zeitnachweis geprüft werden. Diese Bescheidenheit ist keine Schwäche, sondern Voraussetzung für eine belastbare Interpretation.

Als wissenschaftliches Artefakt verbindet das Projekt Datenmodell, Implementierung, Tests, Betrieb und Dokumentation. Die Schnittstellen zwischen Feed, Normalisierung, Codec, Hashing, Speicherung, Manifest und WebUI können einzeln untersucht und erweitert werden. Damit eignet sich News-Hash nicht nur als persönliches Archiv, sondern auch als nachvollziehbarer

Ausgangspunkt für weitere Arbeiten zu digitaler Provenienz, Quellenkritik und langfristiger Nachrichtenüberlieferung.

### **32. Methodische Selbstprüfung**

Ein wissenschaftlicher Bericht über ein laufendes Softwareprojekt muss zwischen Beschreibung, Bewertung und Behauptung unterscheiden. Die Beschreibung bezieht sich auf beobachtbare Funktionen: konfigurierte Quellen, Datensätze, Shards, Hashfelder und Manifeste. Die Bewertung fragt, ob diese Funktionen für das Ziel einer nachvollziehbaren Archivspur geeignet sind. Eine Behauptung über gesellschaftliche Wirkung würde darüber hinaus empirische Daten über Nutzer, Quellen und tatsächliche Archivverwendung benötigen.

Der vorliegende Entwurf nutzt deshalb Szenarien als Prüfraumen. Ein normaler Import prüft die Verarbeitungskette. Ein Wiederholungsabruf prüft die Deduplizierung. Ein Prozessabbruch prüft die unabhängigen Speicherzustände. Eine Veränderung an einem alten Record prüft die Sichtbarkeit eines Integritätsbruchs. Ein beschädigtes Manifest prüft die Trennung zwischen lokalem Inhalt und externem Zeitnachweis. Jedes Szenario besitzt eine erwartete Beobachtung und eine ausdrücklich begrenzte Interpretation.

Diese Form der Selbstprüfung verhindert eine häufige Überdehnung kryptografischer Aussagen. Wenn ein Hash gleich bleibt, bedeutet dies zunächst, dass dieselben Hashinputs dieselbe Digest-Ausgabe erzeugen. Ob der Input vollständig, journalistisch korrekt oder rechtlich speicherbar war, ist eine andere Frage. Die Software kann diese Fragen durch zusätzliche Metadaten unterstützen, aber nicht allein beantworten.

Für eine spätere empirische Studie wären reale Messreihen erforderlich: Feedausfälle über längere Zeit, durchschnittliche Latenzen, Shardwachstum, Wiederherstellungsdauer, Fehlerraten der Codecs und Verständlichkeit der WebUI. Die aktuelle Arbeit ist eine technische Konstruktion mit systematischen Tests, keine Feldstudie. Diese Einordnung ist Teil der wissenschaftlichen Redlichkeit.

### **33. Versionierte Metadaten als Provenienzschrift**

Mit der Einführung von `schema-v1.json`, `codecs-v1.json` und `hash-functions-v1.json` wird die Entstehungsumgebung eines Records expliziter. Ein Record speichert nicht nur Inhalt und Kettenbezug, sondern auch die Hashes der Definitionen, nach denen er verarbeitet wurde. Ein späterer Prüfer kann damit feststellen, ob die zugehörigen JSON-Dateien noch unverändert vorliegen.

Die Versionierung folgt einem append-only Prinzip auf der Ebene der Definitionen. Eine bestehende v0-Datei wird nicht überschrieben, wenn eine neue Codec- oder Schemafassung



entsteht. Stattdessen wird eine v1-Datei abgelegt. Der Record verweist über Version und Hash auf seine Fassung. Diese Regel verhindert, dass eine nachträgliche Änderung der Beschreibung alte Records scheinbar in eine neue technische Umgebung versetzt.

Die Hashes der Metadaten werden selbst nicht rekursiv in die Metadaten aufgenommen. Dadurch entsteht kein unauflöslicher Zirkel. Das Schema beschreibt Recordfelder, der Codec beschreibt Normalisierungsrollen und die Hashfunktionsdefinition beschreibt Kanonisierung und Kettenregel. Der Record nimmt die drei resultierenden Dateihashes auf und verwendet sie in seiner eigenen Digest-Berechnung. Die Abhängigkeit läuft damit in eine Richtung.

Eine spätere Codeänderung muss diese Grenze respektieren. Wird eine Funktion geändert, die das Ergebnis oder die Hashberechnung beeinflussen kann, muss eine neue Definitionsdatei und ein neuer Codecname entstehen. Die Anwendung kann erkennen, ob eine vorhandene JSON-Datei nachträglich verändert wurde; sie kann aber nicht selbst entscheiden, ob eine Quellcodeänderung semantisch eine neue Version verdient. Diese Entscheidung bleibt Teil der Entwicklungspraxis.

### **34. Rückwärtskompatibilität und Migration**

Ein Archiv besteht nicht nur aus der aktuellen Software. Bereits gespeicherte v0-Records müssen auch nach Einführung v1 lesbar und prüfbar bleiben. Deshalb bleiben die alten Codeklassen registriert. Bei der Validierung wird der im Record gespeicherte Codecname verwendet. Alte Records werden mit ihrem ursprünglichen Hashmaterial geprüft; neue Records nutzen die v1-Metadatenfelder.

Die SQLite-Migration erweitert eine vorhandene Records-Tabelle um die neuen Metadatenfelder, sofern das alte erwartete Schema erkannt wird. Alte Zeilen enthalten dort zunächst leere Werte. Ihre ursprüngliche Hashberechnung wird dadurch nicht verändert. Die neue Tabelle erhält zusätzlich eine Artefakttable für Manifest und OTS-Dateien. Eine beliebige fremde oder inkompatible Tabelle wird weiterhin als Legacy-Tabelle gesichert, statt semantisch geraten zu werden.

JSONL benötigt keine tabellarische Migration, weil zusätzliche Felder pro JSON-Objekt eingeführt werden können. Die Validierung behandelt fehlende v1-Felder als Merkmal eines v0-Records, sofern der Codecname dies bestätigt. Ein gemischter Bestand mit alten und neuen Records bleibt damit als eine logische Kette möglich: Der Vorgängerhash verbindet die Records, während die jeweilige Codecdefinition das Hashmaterial bestimmt.

Diese Mischform stellt erhöhte Anforderungen an Werkzeuge. Ein Export darf Metadaten nicht verwerfen. Eine Reparatur darf nicht einfach alle Records mit dem aktuellen Codec neu hashen.

Eine Migration zu einer vollständig einheitlichen v1-Kette wäre ein eigener, ausdrücklich zu protokollierender Vorgang. Im Alltag ist die konservative Lesbarkeit der vorhandenen Daten wichtiger als eine künstliche Gleichförmigkeit.

### **35. Anchor-Artefakte in SQLite**

Manifest und OTS-Proof wurden zunächst als Dateien unter `data/anchors/` geführt. Diese Dateien bleiben für Download und GitHub-Synchronisierung notwendig. Zusätzlich werden beide Binär- beziehungsweise Textinhalte in der Tabelle `anchor_artifacts` des SQLite-Shards gespeichert. Damit liegt der Nachweis nicht nur im Dateisystem, sondern auch in der für die WebUI und Backups relevanten Datenbank.

Die Tabelle verwendet das Paar aus UTC-Datum und Artefakttyp als Schlüssel. Wird für denselben Tag erneut ein Manifest oder Proof gespeichert, wird der Inhalt überschrieben. Diese Regel entspricht der täglichen Anchor-Semantik und verhindert mehrere konkurrierende Kopien innerhalb derselben SQLite-Datei. Der Dateiname wird zusammen mit dem Inhalt gespeichert, damit ein exportiertes Artefakt seine ursprüngliche Rolle behält.

Das Speichern eines Proofs in SQLite verändert nicht die Records-Tabelle und damit nicht die bereits berechneten Nachrichtenhashes. Es erweitert die technische Umgebung des Shards um einen Nachweisdatensatz. Eine Validierung der Nachrichtenkette muss daher die Records-Tabelle prüfen; eine Validierung der Anchor-Artefakte muss zusätzlich Dateisystem, SQLite-Inhalt und gegebenenfalls GitHub-Kopie vergleichen.

Beim Erzeugen eines neuen Anchors werden nach erfolgreichem Stamping beide Dateien in SQLite abgelegt. Anschließend werden die SQLite-Shards in ein Backupverzeichnis kopiert. Das Backup verwendet die stabilen Sharddateinamen und überschreibt die vorherige Kopie. Die Lösung ist bewusst einfach: Sie liefert einen aktuellen Sicherheitsstand, aber keine historische Backupserie. Für eine langfristige Aufbewahrung müssen die überschriebenen Kopien zusätzlich extern versioniert werden.

### **36. Manifestprüfung und gezielte Shards**

Die Validierung berücksichtigt nun nicht nur Records, sondern auch Manifeste. Ein Manifest enthält den letzten Hash und die Nummer des Shards, in dem dieser Hash zu finden ist. Das Prüfwerkzeug öffnet genau diesen JSONL- und SQLite-Shard und vergleicht den dort gefundenen letzten Hash mit dem Manifestwert. Dadurch können auch ältere Tagesmanifeste gegen einen später gewachsenen Archivbestand geprüft werden.

Für große Bestände ist eine gezielte Prüfung sinnvoll. Die Option `--shard N` wählt eine konkrete Shardnummer für beide Speicherformate. Die Prüfung setzt den Vorgängerhash aus dem letzten

Record des vorherigen Shards, kontrolliert aber nur den gewählten Shard selbst. Mit `--all-shards` wird dagegen die gesamte Kette vom Genesiswert an geprüft. Ohne Auswahl wird der letzte nichtleere Shard kontrolliert.

Diese drei Modi dienen unterschiedlichen Fragen. Die letzte-Shard-Prüfung ist schnell und eignet sich für den laufenden Betrieb. Die gezielte Prüfung untersucht einen Manifestbezug oder einen verdächtigen Abschnitt. Die vollständige Prüfung liefert die stärkste lokale Aussage, benötigt aber Zeit proportional zur Zahl der Records und zur Größe der Bilder nicht, da die Bilddaten selbst nicht erneut gehasht werden.

Ein Manifestfehler wird getrennt von einem Recordfehler ausgegeben. Ein fehlender Shard, ein abweichender Endhash oder ein ungültiges JSON-Objekt im Manifest führt zum Fehlerstatus. Die Ausgabe nennt Quelle, Format, Shard und Recordnummer. Damit kann eine Betreiberin nach einem Fehler zunächst den betroffenen Abschnitt sichern, bevor sie weitere Wiederherstellungsmaßnahmen einleitet.

### **37. Backup- und Wiederherstellungsmodell**

Ein Backup ist nur dann aussagekräftig, wenn sein Erzeugungszeitpunkt und sein Inhalt bekannt sind. News-Hash kopiert beim täglichen Anchoring die SQLite-Shards in `data/sqlite-backups/`. Die Namen entsprechen den Quelldateien und werden bei einem neuen Tageslauf überschrieben. Diese Form schützt vor einem kurzfristigen Datenbankverlust, bewahrt aber nicht automatisch die Historie der Backups.

Für eine Wiederherstellung sollte zunächst die Backupdatei auf Lesbarkeit und Schema geprüft werden. Danach ist die Records-Kette zu validieren. Anchor-Artefakte können über den Tages- und Typenschlüssel aus der SQLite-Tabelle gelesen werden. Erst wenn Records, Manifest und OTS-Proof zusammenpassen, sollte eine Kopie wieder als produktiver Speicher eingesetzt werden. Ein bloß erfolgreich geöffnetes SQLite-File ist kein Integritätsnachweis.

Bei mehreren Shards muss die Wiederherstellung die logische Reihenfolge erhalten. Das Kopieren einer einzelnen Datei kann für eine gezielte Diagnose genügen, für einen vollständigen Betrieb aber nicht. Die Shardnummern und die Kettenendpunkte aus dem Manifest helfen, die richtige Datei zu identifizieren. Ein leerer neuer Shard nach einem Rollover ist kein Ersatz für den letzten Shard mit dem manifestierten Hash.

Die Sicherungsstrategie könnte künftig um verschlüsselte Offsite-Kopien, unveränderliche Speichermedien und versionierte Backupmanifeste erweitert werden. Jede zusätzliche Kopie erhöht jedoch die Zahl der Orte, an denen personenbezogene oder urheberrechtlich geschützte

Inhalte liegen. Backupdesign ist daher gleichzeitig eine Integritäts-, Verfügbarkeits- und Datenschutzentscheidung.

### **38. Betriebssicherheit und Heartbeat**

Der Heartbeat hat eine enge Bedeutung: Er signalisiert, dass der Daemon eine neue Meldung verarbeitet hat. Ein laufender Prozess ohne neue Datensätze sendet keinen Ping. Dadurch unterscheidet sich Prozessverfügbarkeit von erfolgreicher Quellenaktivität. Ein Dienst kann technisch erreichbar sein, während eine Quelle leer, unverändert oder fehlerhaft antwortet.

Der Heartbeat wird nach einem erfolgreichen Quellenlauf ausgelöst, wenn mindestens ein Record in JSONL oder SQLite eingefügt wurde. Bei einem Fehler wird kein Erfolgsping gesendet. Werden in einer Runde mehrere Quellen aktualisiert, kann für jede Quelle ein Ping entstehen; die globale Heartbeat-URL erhält weiterhin nur den Statuswert. Eine zukünftige Erweiterung könnte Quelle und Recordanzahl im Payload ergänzen, müsste aber die Erwartungen bestehender Monitoringdienste berücksichtigen.

Für die Interpretation eines fehlenden Pings sind zusätzliche Metriken erforderlich. Prometheus kann Quellenfehler und letzte erfolgreiche Verarbeitung darstellen. Ein externes Monitoring sollte zwischen „Prozess ist nicht erreichbar“, „Quelle liefert keine neuen Meldungen“ und „Quelle meldet einen Fehler“ unterscheiden. Der Heartbeat allein ist kein Beweis für Datenintegrität und kein Ersatz für die Hashkettenvalidierung.

Diese enge Semantik reduziert Fehlalarme bei Quellen, die nur selten aktualisiert werden. Gleichzeitig muss für jede Quelle ein plausibles erwartetes Intervall bekannt sein. Eine tägliche Meldungsquelle darf nicht wie ein minütlicher Feed überwacht werden. Die Konfiguration bildet daher nicht nur den Abrufplan, sondern auch den Interpretationsrahmen für Betriebsereignisse.

### **39. Benutzeroberfläche als Prüfwerkzeug**

Die WebUI ist nicht nur eine Anzeige der neuesten Nachrichten. Sie bildet die Übergänge zwischen Archivdaten und Prüfprozess ab. Eine Quellenkarte zeigt Anchor-Status und Links zu Manifest und Proof. Die Meldungsdetailseite zeigt Hash und Vorgängerhash. Die Laufzeitinformationen nennen Fehler und Abrufereignisse. Diese Elemente ermöglichen eine schrittweise Untersuchung, ohne dass jeder Nutzer direkt SQLite öffnen muss.

Für eine verständliche Validierung wären zusätzliche Links zur Sharddatei und zur Prüfkation sinnvoll. Ein Nutzer könnte aus einer Manifestansicht direkt die zugehörige Shardnummer sehen und die gezielte Prüfung starten. Dabei müsste die WebUI klar zwischen einem Download, einer lokalen Prüfung und einer externen OTS-Verifikation unterscheiden. Eine Schaltfläche mit dem

Wort „verifiziert“ wäre nur zulässig, wenn bekannt ist, welche Ebenen tatsächlich geprüft wurden.

Die Themen und die responsive Darstellung sind für die technische Integrität nicht entscheidend, beeinflussen aber die Zugänglichkeit. Ein Hash, der nur in einer überfüllten Desktopansicht sichtbar ist, wird praktisch schwerer geprüft. Die Oberfläche sollte lange Werte umbrechen, Kopieren ermöglichen und Fehlermeldungen mit Quelle, Format und Shard ausgeben. Gestaltung ist hier eine Vermittlungsleistung zwischen kryptografischer Präzision und menschlicher Aufmerksamkeit.

Die englische Dokumentationsseite übersetzt die Erklärtexte, während die Dashboard-Oberfläche deutsch bleibt. Diese Trennung muss auch bei neuen Validierungs- und Manifestfunktionen berücksichtigt werden. Technische Feldnamen wie `latest_shard_jsonl` bleiben stabil, erklärende Hinweise können sprachabhängig ergänzt werden. Eine API- oder Exportdarstellung sollte nicht von sichtbaren Übersetzungen abhängen.

#### **40. Ergebnis und Ausblick**

Die Erweiterungen aus den beiden Aufgabenlisten verbinden Archivierung, Versionskontrolle und Betrieb stärker miteinander. Versionierte JSON-Dateien machen sichtbar, nach welchen Regeln ein Record erzeugt wurde. Neue Hashfelder binden diese Regeln kryptografisch an die Nachricht. SQLite enthält nun zusätzlich die Nachweise selbst und erhält beim Anchoring eine überschreibbare Sicherung. Die Validierung kann Records, Manifeste und konkrete Shards in einem gemeinsamen Ablauf kontrollieren.

Diese Verbindung reduziert eine wichtige Mehrdeutigkeit älterer Archive. Ein Hash ohne Beschreibung seines Schemas ist nur eingeschränkt reproduzierbar. Ein Manifest ohne Shardnummer ist für große Bestände schwer zuzuordnen. Ein Proof außerhalb der Datenbank kann verloren gehen, obwohl der Recordbestand erhalten bleibt. Die neuen Metadaten und Artefakttabellen lösen nicht alle Archivprobleme, machen aber die relevanten Beziehungen expliziter.

Die nächsten Schritte sollten sich auf unabhängige Verifikation, sichere Backups und verständliche Nutzerführung konzentrieren. Ein separates Tool könnte Metadatenversion, Codecdefinition, Recordkette, Manifest und OTS-Proof ohne Netzwerkzugriff prüfen. Ein formales Backupformat könnte mehrere SQLite-Shards in einer unveränderlichen Einheit bündeln. Die WebUI könnte den Prüfstatus mit klarer Herkunft und Zeitangabe darstellen.

Das Motto bleibt dabei passend: „Mit KI gebaut, mit Liebe verfeinert, für neugierige Menschen.“ Die KI-Unterstützung beschleunigt Konstruktion und Dokumentation, ersetzt aber weder

Versionsentscheidungen noch Tests noch Quellenkritik. Liebe zur Ausführung zeigt sich in klaren Namen, reproduzierbaren Prüfungen und ehrlichen Grenzen. Neugier bedeutet, die Meldung, ihre Herkunft, ihre Speicherung und ihre technische Beweislage gemeinsam zu betrachten.

---

## Literatur und Projektdokumentation

- [1] News-Hash-Projekt: *Architektur*, `project-docu/architecture.md`, Projektstand 2026.
- [2] News-Hash-Projekt: *Anforderungen*, `project-docu/requirements.md`, Projektstand 2026.
- [3] News-Hash-Projekt: *README und WebUI-Dokumentation*, Projektstand 2026.
- [4] News-Hash-Projekt: *Architekturentscheidungen*, `project-docu/decisions.md`, Projektstand 2026.
- [5] News-Hash-Projekt: *WebUI*, `project-docu/webui.md`, Projektstand 2026.
- [6] Consultative Committee for Space Data Systems: *Reference Model for an Open Archival Information System (OAIS)*, CCSDS 650.0-M-2, 2012. <https://public.ccsds.org/Pubs/650x0m2.pdf>.
- [7] Groth, P.; Moreau, L. (Hrsg.): *PROV-Overview: An Overview of the PROV Family of Documents*, W3C Working Group Note, 2013. <https://www.w3.org/TR/prov-overview/>.
- [8] Van de Sompel, H.; Nelson, M.; Sanderson, R.: *HTTP Framework for Time-Based Access to Resource States - Memento*, RFC 7089, IETF, 2013. <https://www.rfc-editor.org/rfc/rfc7089>.
- [9] National Institute of Standards and Technology: *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. <https://doi.org/10.6028/NIST.FIPS.180-4>.
- [10] OpenTimestamps: *A timestamping proof standard*, <https://opentimestamps.org/>.

Dieser Artikel beschreibt den Projektstand vom 17. August 2026. Die PDF wurde aus dieser HTML-Quelle mit Chromium im A4-Format erzeugt.